

آموزش Call File

AUTO DIALER



عناوین این آموزش

- معرفی فایل‌های داخل پکیج آموزشی
- معرفی ابزارها و برنامه‌های استفاده شده در این آموزش
- معرفی Call File ها و انواع جریان های اجرایی در آن
- معرفی کاربردهای Call File
- ساختار یک Call File
- نکات و TRBL در کار با Call File
- بررسی دستورات محیط لینوکس مورد نیاز
- پیاده سازی یک مثال ساده با دو جریان اجرایی
- بررسی یک سناریو – ساخت ماژول تماس خودکار
- شرح سناریو
- ساخت Call File اصلی با دو جریان اجرایی
- برنامه نویسی PHP

معرفی فایل‌های داخل پکیج آموزشی

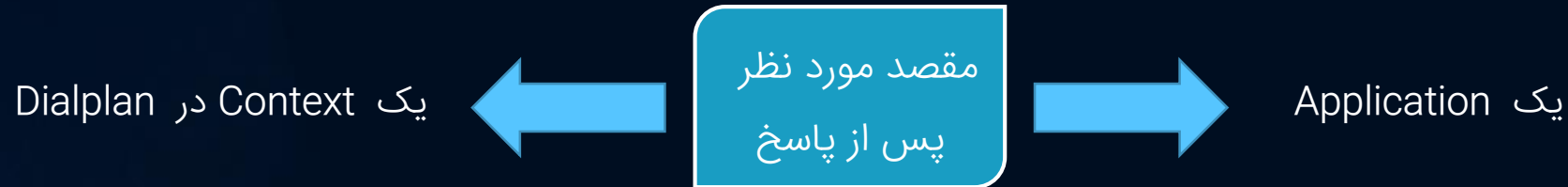
- سه فایل mp4 از محتویات دوره
- یک فایل Powr Point از محتویات دور
- یک فایل PDF از محتویات دوره
- یک فایل PHP که سناریوی اصلی در آن کد نویسی شده است
- یک فایل TXT که در پیاده سازی سناریو از آن استفاده شده است
- دو فایل Call که نمونه‌ای از کال فایل‌ها می‌باشد
- یک فایل TXT که کدهای Dialplan در آن قرار دارد

معرفی برنامه‌های استفاده شده در این آموزش

- برنامه WinSCP برای دسترسی آسان به فایل‌های سرور ویپ
- برنامه PuTTY برای اتصال به کنسول لینوکس و استریسک
- برنامه Notepad ++ برای ویرایش فایل‌ها و کدنویسی سناریو

معرفی Call File

هدف اصلی یک Call file ایجاد یک کانال به صورت اتوماتیک و اتصال آن پس از پاسخ به مقصد مورد نظر می باشد .



- پسوند یک call file «call» می باشد .
- استریسک به محض مشاهده یک Call File در مسیر زیر , اقدام به اجرای آن می نماید .
`/var/spool/asterisk/outgoing/`
- ماژول pbx_spool.so وظیفه مراقبت از مسیر بالا را دارد و به محض رویت یک فایل , شروع به پردازش آن فایل می نماید .

معرفی Call File

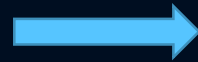
دایرکتوری های مربوط به call files :

/var/spool/asterisk/outgoing/



مسیر اجرا

/var/spool/asterisk/outgoing_done/



مسیر فایل های آرشیو شده

معرفی کاربردهای Call File

- ایجاد کمپین‌های تبلیغاتی
- اجرای برنامه‌های زمانبندی شده در سرور ویپ : پخش اذان در تلفن‌ها و یا بلندگوها به هنگام اذان
- ایجاد برنامه‌های یادآوری در زمان مشخص : یادآوری زمان جلسات به شرکت‌کنندگان در جلسه
- ایجاد تعداد دلخواه از تماس‌های هم‌زمان در سرور ویپ

ساختار Call File

<Key>: <value>

Channel: <channel>	مقصد
Callerid: <callerid>	کالرآیدی تماس
WaitTime: <number>	تعداد ثانیه های انتظار برای پاسخ گرفتن – زمان پیش فرض ۴۵ ثانیه
MaxRetries: <number>	تعداد دفعات تلاش در صورت عدم دریافت پاسخ – دفعات پیش فرض ۵ بار می باشد
RetryTime: <number>	تعداد ثانیه های انتظار قبل از سعی مجدد – زمان پیش فرض ۳۰۰ ثانیه می باشد
Archive: <yes no>	نتیجه آرشیو شود یا خیر ؟

Application: <appname>	نام برنامه ای که قرار است اجرا شود
Data: <args>	آرگومان های برنامه مورد نظر

Context: <context>	نام کانتکست مورد نظر
Extension: <exten>	اکستنشن مورد نظر
Priority: <priority>	اولویت مورد نظر – هم عدد و هم لیبل
Setvar: <var=value>	مقدار دهی به یک متغیری که قرار است در کانتکست مورد نظر از آن استفاده شود

ساختار Call File

استریسک نتایج زیر را به فایل آرشیو شده اضافه می نماید :

Status: <exitstatus> : "Expired", "Completed" , "Failed"

StartRetry : <pid> <retrycount> (<time>)

EndRetry : <pid> <retrycount> (<time>)

حداقل خطوط در یک کال فایل :

Channel: SIP/۴۰۱

Application: Playback

Data: hello-world



ساختار Call File

جهت کامنت گذاری در Call File ها از کاراکتر '#' و ';' در ابتدای خطوط فایل می توان استفاده کرد .

Channel: sip/trunk1/09121716637

Callerid: 22902602

wait for 5 second to get answer

WaitTime: 5

MaxRetries: 2



این خط یک کامنت می باشد .

ساختار Call File

چنانچه از جریان اجرایی Dialplan استفاده می‌کنید و نتیجه تماس کال فایل به هر دلیلی موفقیت آمیز نباشد ، استریسک کانال را به **extension** ای به نام **failed** در همان **context** داده شده در کال فایل ، منتقل می‌نماید و متغییری با نام **\${REASON}** را مقدار دهی می‌کند . شما می‌توانید از این امکان برای لاگ برداری و یا ارسال اطلاعات به جدول CDR و یا هر دیتابیزی استفاده نمایید .

Call File :

```
channel: sip/trunk_۱/۰۹۱۲۱۲۳۴۵۶۷
context: ads
extension: ۱۱۰
Priority: ۱
```

Dialplan :

```
[ads]
exten => ۱۱۰ , ۱ , noop(start the ads)
      same => n , wait(۳)
      same => n , playback(hello-world)
      same => n , hangup()
exten => failed , ۱ , noop(call file failed)
      same => n , noop(${REASON})
```

نکات و TRBL در کار با Call File

۱. هیچ گاه و هیچ گاه call file ها را در مسیر outgoing به طور مستقیم ایجاد نکنید .
۲. هیچ گاه و هیچ گاه call file ها را در مسیر outgoing کپی نکنید .
۳. برای عملکرد صحیح call file ها همواره آن ها را در یک مسیر دیگر در همان سیستم فایل ایجاد نمایید و سپس از آنجا به مسیر outgoing منتقل نمایید . (عمل move)
`mv /tmp/test.call /var/spool/asterisk/outgoing`
۴. قبل از عمل move مطمئن شوید فایل دسترسی write را دارد .
۵. قبل از عمل move مطمئن شوید owner فایل یوزر asterisk می باشد . (برای نوشتن زمان شروع و پایان ----> عمل utime)
۶. هیچ گاه و هیچ گاه call file ها را با کدینگ utf8 و یا unicode ذخیره ننمایید .
۷. متأسفانه در خطوط آنالوگ سیستم استریسک به محض دریافت بوق آزاد تماس را پاسخ داده شده در نظر میگیرد و شروع به انجام اقدام مورد نظر می نماید .
۸. به جهت بازدهی بهتر در خطوط آنالوگ بهتر است از جریان dialplan استفاده نمایید و در آن حتما از دستور wait نیز استفاده نمایید .

معرفی دستورات محیط لینوکس

- `chmod ۶۴۴ /path/of/file`

نوع دسترسی فایل

- `touch -t ۲۰۱۷۰۱۰۵۱۰۰۰ /path/of/file`

زمان مجاز برای دسترسی به فایل

- `chown asterisk:asterisk /path/of/file`
- `mv path/of/file new/path`
- `php path/of/file`
- `echo "sometext" >> /path/of/file`

پیاده سازی سناریو

استریسک در یک **زمان مشخص** به صورت **اتوماتیک** با یک **گروه از مشتریان** ما تماس می گیرد و یک متن تبلیغاتی را برای آنها پخش می نماید .
استریسک شماره مخاطبان را از یک فایل TXT به نام numbers.txt از پوشه tmp خواهد خواند .
استریسک ۱۰ ثانیه برای پاسخ دهی هر مشتری منتظر می ماند .
استریسک در صورت **پاسخ گویی** مشتری شماره آن مشتری را در یک فایل TXT به نام success.txt در پوشه tmp ذخیره خواهد کرد .
استریسک در صورت **عدم پاسخ گویی** مشتری شماره آن مشتری را در یک فایل TXT به نام failed.txt در پوشه tmp ذخیره خواهد کرد .
استریسک تماس های مورد نظر را آرشیو می نماید .

با تمرین زیاد به آسانی یاد بگیریم

VoiPing.ir

